Figure 16.4 Using model binding to create an instance of a model that's used to execute a Razor Page

The `Id` property has been bound from a URL route parameter, but the `Name` and `SellPrice` properties have been bound from the request body. The big advantage of using model binding is that you don't have to write the code to parse requests and map the data yourself. This sort of code is typically repetitive and error-prone, so using the built-in conventional approach lets you focus on the important aspects of your application: the business requirements.

TIP Model binding is great for reducing repetitive code. Take advantage of it whenever possible, and you'll rarely find yourself having to access the `Request` object directly.

If you need to, the capabilities are there to let you completely customize the way model binding works, but it's relatively rare that you'll find yourself needing to dig too deep into this. For the majority of cases, it works as is, as you'll see in the remainder of this section.

16.2.1 Binding simple types

We'll start our journey into model binding by considering a simple Razor Page handler. The next listing shows a simple Razor Page that takes one number as a method parameter and squares it by multiplying the number by itself.

Listing 16.2 A Razor Page accepting a simple parameter

```
public class CalculateSquareModel : PageModel
{
    public void OnGet(int number)       ❶
    {
        Square = number * number;       ❷
    }

    public int Square { get; set; }     ❸
}
```